



UNIVERSIDADE KIMPA VITA
INSTITUTO POLITÉCNICO

LÓGICA DE PROGRAMAÇÃO

RELATORIO PRATICO DE LÓGICA DE PROGRAMAÇÃO

GRUPO Nº 4

SISTEMA DE GESTÃO DE ESTUDANTES

Implementado em Linguagem C

Nº	Nome dos Membros
1	Nsessa Sebastião Manzambi
2	Manuel Alberto
3	Horácio Joaquim Zoca
4	João António Ambrosio
5	Angelo Antonio Nicolau

Uíge, 2005/2026

1. Introdução

O presente relatório documenta o desenvolvimento do projecto prático da disciplina de Lógica de Programação, cujo objectivo central foi a concepção e implementação de um Sistema de Gestão de Estudantes em linguagem C. Este projecto foi elaborado com o propósito de consolidar e aplicar os conhecimentos adquiridos ao longo da unidade curricular, abrangendo estruturas de dados, algoritmos de ordenação e pesquisa, manipulação de ficheiros e interacção com o utilizador através de menus.

O sistema permite armazenar, visualizar, ordenar, pesquisar, inserir e gravar informações sobre estudantes, nomeadamente matrícula, nome e três notas, calculando automaticamente a média de cada aluno.

2. Objectivos do Projecto

2.1 Objectivo Geral

Desenvolver um sistema completo de gestão de estudantes em C, demonstrando domínio dos conceitos fundamentais da lógica de programação.

2.2 Objectivos Específicos

- Implementar estruturas (struct) para representar dados de estudantes
- Ler e gravar dados em ficheiros de texto e binários
- Aplicar três algoritmos de ordenação: Bubble Sort, Selection Sort e Insertion Sort
- Implementar pesquisa linear e binária por matrícula e por nome
- Permitir inserção manual de estudantes via teclado
- Construir um menu interactivo para navegação no sistema

3. Estrutura do Projecto

O projecto está organizado em três ficheiros principais, seguindo boas práticas de modularização em C:

Ficheiro	Tipo	Descrição
main.c	Ficheiro Principal	Ponto de entrada do programa; contém o menu principal e o loop do sistema
funcoes.h	Cabeçalho	Define a estrutura Estudante, constantes (MAX_ESTUDANTES, MAX_NOME) e os protótipos de todas as funções
FuncProntas.c	Implementação	Contém a implementação de todas as funções: leitura, exibição, ordenação, pesquisa, inserção, editar, eliminar, e gravação

3.1 Estrutura de Dados

A estrutura central do sistema é definida em funcoes.h:

```
typedef struct Estudante {  
    int    matricula;           // Número de matrícula  
    char   nome[100];          // Nome completo  
    float  nota1;               // Primeira nota  
    float  nota2;               // Segunda nota  
    float  nota3;               // Terceira nota  
    float  media;               // Média calculada automaticamente  
} Estudante;
```

4. Funcionalidades Implementadas

4.1 Leitura de Ficheiro

A função `lerFicheiro()` abre o ficheiro `estudantes.txt` no formato `Matrícula;Nome;Nota1;Nota2;Nota3`, lê cada linha com `fscanf()` e calcula automaticamente a média das três notas para cada estudante carregado.

4.2 Exibição dos Estudantes

A função `exibirEstudantes()` apresenta todos os estudantes em memória num formato tabular alinhado, exibindo matrícula, nome, as três notas e a média calculada.

4.3 Ordenação

O sistema disponibiliza três algoritmos de ordenação clássicos, com três critérios de ordenação cada:

Algoritmo	Descrição
Bubble Sort	Compara e troca pares adjacentes repetidamente até a lista estar ordenada
Selection Sort	Selecciona o menor elemento e coloca-o na posição correcta em cada iteração
Insertion Sort	Insere cada elemento na posição correcta de uma sublista já ordenada

Critérios disponíveis: por Matrícula (crescente), por Nome (ordem alfabética) e por Média (decrescente).

4.4 Pesquisa

O módulo de pesquisa oferece dois métodos distintos:

- Pesquisa Linear por Matrícula — percorre todos os elementos sequencialmente; funciona com qualquer lista.
- Pesquisa Binária por Matrícula — algoritmo de divisão e conquista; requer que a lista esteja ordenada por matrícula.
- Pesquisa Linear por Nome — compara nomes com `strcasecmp()`, ignorando maiúsculas/minúsculas.

4.5 Inserção Manual

A função `inserirEstudanteTeclado()` permite ao utilizador introduzir manualmente os dados de um novo estudante (matrícula, nome e três notas). A média é calculada automaticamente e o estudante é adicionado à lista em memória.

4.6 Gravação em Ficheiro

A função `gravarFicheiro()` permite salvar os dados actualizados em dois formatos:

- Ficheiro de Texto (.txt) — dados separados por ponto e vírgula, legível por qualquer editor de texto.
- Ficheiro Binário (.bin) — gravação directa da estrutura em memória com `fwrite()`, mais eficiente para leitura posterior pelo programa.

5. Manual de Utilização

5.1 Compilação do Projecto

O projecto utiliza um Makefile para automatizar a compilação. No terminal (ou na IDE Dev-C++), execute:

```
make
```

Ou compile manualmente com:

```
gcc -o sistema.exe main.c FuncProntas.c -I.
```

5.2 Menu Principal

Ao executar o programa, é apresentado o seguinte menu interactivo:

```
=====
      SISTEMA DE GESTAO DE ESTUDANTES
=====
[1] Ler Ficheiro de Estudantes
[2] Exibir Estudantes em Memoria
[3] Ordenar Lista de Estudantes
[4] Pesquisar por Estudante
[5] Gravar Dados Atualizados em Ficheiro
[6] Inserir Estudante Manualmente (Teclado)
[0] Sair do Sistema
=====
Escolha uma opcao:
```

5.3 Passo a Passo de Utilização

Opção 1 — Ler Ficheiro de Estudantes

Antes de qualquer outra operação, o utilizador deve carregar os dados. O ficheiro estudantes.txt deve estar na mesma pasta do executável, com o seguinte formato (um estudante por linha):

```
20240001;Ana Silva;15.0;14.5;16.0
20240002;Bruno Costa;12.0;11.5;13.0
20240003;Carla Mendes;18.0;17.0;19.5
```

Após a leitura, o sistema confirma: "Dados carregados com sucesso! N estudantes em memória."

Opção 2 — Exibir Estudantes em Memória

Mostra todos os estudantes carregados em formato tabular com matrícula, nome, notas (N1, N2, N3) e média calculada. Se nenhum ficheiro foi lido, o sistema avisa o utilizador.

Opção 3 — Ordenar Lista de Estudantes

1. Escolha o algoritmo: 1 = Bubble Sort, 2 = Selection Sort, 3 = Insertion Sort
2. Escolha o critério: 1 = Matrícula, 2 = Nome, 3 = Média

A lista em memória é reordenada e uma mensagem de confirmação é apresentada.

Opção 4 — Pesquisar por Estudante

3. Escolha o tipo de pesquisa: 1 = Matrícula, 2 = Nome
4. Se por matrícula: escolha entre Pesquisa Linear (qualquer lista) ou Binária (lista ordenada por matrícula)

5. Introduza o valor a pesquisar

Se encontrado, os dados completos do estudante são exibidos. Caso contrário, o sistema informa que o estudante não foi encontrado.

ATENÇÃO: A pesquisa binária só produz resultados correctos se a lista estiver previamente ordenada por matrícula.

Opção 5 — Gravar Dados em Ficheiro

6. Escolha o formato: 1 = Texto (.txt) ou 2 = Binário (.bin)

7. Introduza o nome do ficheiro (sem extensão, ex: atualizado)

O ficheiro é criado na pasta do executável com todos os dados actuais em memória.

Opção 6 — Inserir Estudante Manualmente

8. Introduza o número de matrícula

9. Introduza o nome do estudante

10. Introduza as três notas individualmente

A média é calculada automaticamente e o estudante é adicionado à lista em memória. Para tornar a inserção permanente, utilize a Opção 5 para gravar o ficheiro.

Opção 0 — Sair do Sistema

Encerra o programa. Atenção: os dados em memória que não tiverem sido gravados serão perdidos.

6. Conceitos de Lógica de Programação Aplicados

Conceito	Aplicação no Projecto
Estruturas (struct)	Encapsula matrícula, nome, notas e média numa única entidade
Arrays de estruturas	Lista de até 200 estudantes em memória (baseDados[MAX_ESTUDANTES])
Ponteiros	Passagem por referência (&totalEstudantes) para modificar variáveis externas
Funções modulares	Código organizado em funções independentes e reutilizáveis
Ficheiros (FILE*)	Leitura com fscanf/fgets e gravação com fprintf/fwrite
Algoritmos de ordenação	Bubble, Selection e Insertion Sort implementados do zero
Pesquisa linear e binária	Duas abordagens com análise de complexidade implícita
Strings em C	Manipulação com strcmp, fgets, strtok e strcat
Pré-processador	#define, #ifndef / #endif para constantes e include guard
Ciclos e condicionais	do-while no menu, for nos algoritmos, while na leitura de ficheiro

7. Dificuldades Encontradas e Soluções

7.1 Erro de Ligação (Linker Error)

Durante o desenvolvimento, ocorreu um erro crítico de compilação: `undefined reference to 'inserirEstudanteTeclado'`. A causa foi a definição da função `inserirEstudanteTeclado` dentro do corpo da função `gravarFicheiro`, uma vez que a linguagem C não permite funções aninhadas.

Solução: A função foi extraída e colocada no nível global do ficheiro `FuncProntas.c`, após o fecho correcto da função `gravarFicheiro`.

7.2 Gestão do Buffer de Entrada

Ao misturar `scanf()` com `fgets()`, o caractere de nova linha residual no buffer causava leituras incorrectas de strings. A solução foi inserir `getchar()` após chamadas a `scanf()` que precedem `fgets()`, limpando o buffer antes da leitura do nome.

7.3 Pesquisa Binária com Lista Não Ordenada

A pesquisa binária retornava resultados incorrectos quando aplicada a uma lista não ordenada por matrícula. A solução foi incluir um aviso explícito no menu, informando o utilizador de que a lista deve estar previamente ordenada por matrícula para obter resultados correctos.

8. Conclusão

O presente projecto permitiu consolidar de forma prática e integrada os principais conceitos da disciplina de Lógica de Programação em C. A implementação do Sistema de Gestão de Estudantes exigiu a aplicação articulada de estruturas de dados, algoritmos clássicos, manipulação de ficheiros e boas práticas de modularização do código.

O grupo concluiu que a organização do código em múltiplos ficheiros (`main.c`, `funcoes.h` e `FuncProntas.c`) facilita a manutenção, a leitura e a depuração do programa. Os erros encontrados durante o desenvolvimento — nomeadamente o erro de ligação causado por função aninhada — foram valiosas oportunidades de aprendizagem sobre o funcionamento interno do compilador e do linker em C.

O sistema encontra-se funcional e cumpre todos os requisitos propostos, demonstrando que os membros do grupo assimilaram os conteúdos da disciplina com sucesso.

9. Referências Bibliográficas

- KERNIGHAN, Brian W.; RITCHIE, Dennis M. — The C Programming Language. 2.^a ed. Prentice Hall, 1988.
- CORMEN, Thomas H. et al. — Introduction to Algorithms. 3.^a ed. MIT Press, 2009.
- Documentação oficial do GCC — <https://gcc.gnu.org/onlinedocs/>
- Apontamentos e materiais da disciplina de Lógica de Programação — 2025